Testing for Deriving Observable State of Software Systems

Evidence Paths, Bounded Proof Artifacts, and Constitutional Verification

Software tests commonly verify selected behavior under controlled conditions.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Software tests commonly verify selected behavior under controlled conditions. Production operation poses another question: can responsible observers determine what happened, when, under which authority, with which effects, and within what uncertainty? A system can pass behavioral tests while its receipts, event transport, storage, derivations, health indicators, or observation surfaces are missing, stale, corrupted, misleading, or inaccessible.

This paper develops Testing as Observability Expansion within the proposed DSS:OS:TEST domain. It treats a test as a bounded evidence artifact supporting a declared claim under a scenario, environment, oracle, observation regime, and inference range. Tests do not generally prove that software "can be known"; they can test specified links in an evidence path and reveal untested or failed regions.

The revised architecture replaces an absolute evidence hierarchy with a dependency graph, treats seven evidence classes as overlapping facets, defines an evidence-path contract and a Constitutional Test Runner report, and makes observability closure relative to a declared claim inventory and profile. It integrates witness semantics, receipt-based reasoning, epistemic situational awareness, and meta-binding tests while preserving their distinct roles.

Source, Evidence, and Standing

This is an AI-assisted integrated edition derived from one exact author manuscript. Its custody and comparison manifest is
`work/paper-saturation-v1/evidence/testing-for-deriving-observable-state/integration-custody-manifest.v0.1.json`,
SHA-256

`588B20534ABF7102FFADA9EA45ABDD7DDAA04FA35A9A15F31361FC0ECBEAEF35`.
The source remains unchanged.

The DSS hierarchy, evidence layers, test classes, runners, closure concept, examples, and core thesis are author material. Formal qualifications, role separation, countertests, and decisions are AI editorial or synthetic candidates until adopted. No DSS conformance or deployment readiness is established.

1. Behavior and Knowability

A behavioral claim and an observability claim concern different things:

- **Behavioral claim:** under condition q, the subject produced outcome y.
- **Observability claim:** under observation regime o, an authorized observer can obtain evidence sufficient to assess a declared property of that outcome within scope and time bounds.

Neither implies the other. A system may behave correctly and emit no useful evidence. It may emit impeccable telemetry while behaving incorrectly. It may also produce evidence that is complete syntactically but false, spoofed, selectively unavailable, or misinterpreted.

2. Bounded Support, Not Unlimited Proof

Represent a test artifact as support for claim c:

$$t \vdash_{e,o,p} c,$$

where e is the test environment and scenario, o the oracle and observation regime, and p the profile defining scope, evidence requirements, and inference limits. The symbol denotes bounded support, not mathematical proof unless a separate formal proof system justifies that term.

The result should state what was observed, what inference is licensed, what was not tested, and what production differences can defeat transfer.

3. Evidence Path

An observable-state claim depends on a path:

$$
\begin{aligned}
&\mathit{event}\\
&\rightarrow \mathit{emission}\\
&\rightarrow \mathit{capture}\\
&\rightarrow \mathit{transport}\\
&\rightarrow \mathit{storage}\\
&\rightarrow \mathit{derivation}\\
&\rightarrow \mathit{surface}\\
&\rightarrow \mathit{interpretation}
\end{aligned}
.
$$

Each link has identity, timing, loss, ordering, duplication, integrity, access, privacy, and failure properties. End-to-end tests should be complemented by link tests and independent checks. A green dashboard can coexist with dropped events; a valid receipt chain can faithfully preserve false declarations.

The path contract records subject, event class, emitter, schema, clock, transport, persistence, derivation, display, access, retention, expected latency, failure signal, and responsible custodian.

4. Evidence Dependency Graph

Raw receipts, ESA assessments, observation surfaces, and tests are not necessarily a simple ascending hierarchy. Tests consume or generate evidence; surfaces may display test results; assessments may trigger tests; receipts may record both system and test actions.

Model them as a dependency graph with typed epistemic roles:

- **event evidence** records a declared observation or operation;
- **derived assessment** applies a declared inference to evidence;
- **observation surface** presents selected evidence and derivations;
- **test evidence** records a controlled challenge and observed response;
- **independent witness** checks selected claims through a different path.

Every node carries source, method, time, scope, uncertainty, standing, and dependencies. Cycles must be exposed rather than mistaken for independent corroboration.

5. Test Artifact Contract

A DSS:OS:TEST artifact should include:

- immutable test and version identity;
- subject, subsystem, invariant, and claim bindings;
- scenario, preconditions, fixtures, and perturbations;
- environment and production-difference model;
- oracle, thresholds, and expected outcomes;

- evidence-path links under test;
- evidence-class facets;
- observations, raw outputs, hashes, and custody;
- result, uncertainty, nondeterminism, and retries;
- inference range and claim ceiling; and
- author, authority, review, expiration, and revision lineage.

A test script alone is not the proof artifact. The artifact is the bound test execution and its interpretable evidence.

6. Evidence-Class Facets

Structural, behavioral, governance, integration, performance, environmental, and negative evidence are useful facets. They need not form a lattice or mutually exclusive classes. One scenario may carry several facets, and one invariant may require multiple independent forms.

Coverage counts should not substitute for adequacy. A thousand trivial governance tests may cover less risk than one well-designed authority-bypass scenario. Reports should include claim criticality, independence, environmental reach, oracle strength, and mutation or fault sensitivity.

7. Constitutional Test Runner

The proposed Constitutional Test Runner (CTR) can bind executions to claims and report:

- claims and invariants with no evidence;
- evidence paths with untested links;
- class and risk distribution;
- stale, expired, quarantined, or flaky results;
- production differences outside inference range;
- conflicting or circular evidence;
- bypass and negative-test coverage;
- independent witness status; and
- unresolved failures and waivers.

CTR output is an assessment under its own profile. It is not self-validating. The runner requires a trusted base, reproducible fixtures, versioned policy, negative controls, and where risk warrants, independent or diverse verification.

Testing CTR with CTR can reveal regressions but does not end regress. Bootstrap checks must identify which assumptions remain outside the tested system.

8. Governed Scenario Execution

A HeadlessRunner can execute controlled scenarios and collect behavioral outcomes, receipt deltas, stream-integrity results, health observations, ESA assess-

ments, and governance snapshots. Its role is evidence capture, not automatic truth.

`assert_clean()` should expand into named conditions and thresholds. A single clean flag can hide whether behavior passed, a receipt was present, chain integrity held, health was fresh, authority remained bounded, side effects corresponded, or an assessment was merely below threshold.

Use a result vector such as:

$$r = (behavior, evidence, integrity, freshness, authority, effect, recovery, unknown).$$

Typed results preserve cases such as behaviorally correct but evidence-dark, receipt-valid but effect-mismatched, or assessment-clean but outside inference range.

9. Relative Observability Closure

Absolute closure would require knowing that every relevant observable-state claim has been enumerated and validly tested, including unknown failure modes. The achievable construct is **profile-relative observability coverage**.

Let C_p be a declared claim inventory and $V_p \subseteq C_p$ the claims with current, adequate, non-quarantined support. Then:

$$Coverage_p = \frac{\sum_{c \in V_p} w(c)}{\sum_{c \in C_p} w(c)}.$$

The report must separately disclose inventory completeness, unmodeled regions, dependency independence, environment transfer, and critical unsupported claims. A numerical score never overrides a critical gap.

10. Test-to-Production Inference

Test results transfer only within declared adequacy across data, environment, concurrency, timing, scale, failure, dependency, security, access, recovery, and human-operation shapes. Production can invalidate assumptions through configuration drift, partial failure, clock behavior, sampling, load, permissions, network partition, or observer unavailability.

Deployment evidence should therefore combine predeployment tests with canaries, runtime monitors, fault injection where authorized, incident exercises, recovery tests, and postdeployment witnesses. Tests expand observability; they do not replace production observation.

11. Deployment Readiness

A system that works but cannot be diagnosed may be operationally unacceptable. Yet observability alone does not establish deployability. Readiness also depends on correctness, safety, security, privacy, accessibility, performance, recovery, maintainability, authority, operations, and context-specific obligations.

DSS:OS:TEST should contribute an evidence-readiness verdict to a larger decision. It should return
`READY_WITHIN_PROFILE`,
`QUALIFIED`, `HOLD`, `REFUSE`, or `NOT_COVERED`,
with reasons and unresolved risks, rather than declare deployment from closure alone.

12. Corpus Integration and Authorial Compression

Testing for Deriving Observable State specifies test evidence for evidence paths and observability claims. **Governance Witnesses** specifies what observations and witnesses can support. **Reasoning by Way of Receipts** specifies how bounded inference proceeds from receipted artifacts. **Epistemic Situational Awareness** derives higher-order assessments. **Project57** specializes testing of meta-binding preservation across transformation.

These papers form a chain but not one identity. Witnesses do not guarantee truth; receipts do not guarantee effects; ESA does not prove cause; tests do not certify production; Project57 does not cover all observability.

The **Governed Meaning Base (GMB)** can retain claim inventories, test artifacts, evidence paths, results, waivers, incidents, and revisions. The universal **GSM** can type supports, observes, derives-from, contradicts, expires, covers, and leaves-unknown relations without promoting them to proof.

13. Realization Funnel

$$
\begin{aligned}
&\mathit{Possibility} \\
&\to \mathit{Specification} \\
&\to CM_1 \\
&\to CM_2 \\
&\to IC \\
&\to IR \\
&\to \mathit{Evidence} \\
&\to \mathit{Revision}
\end{aligned}
$$

- **Possibility:** enumerate unit, integration, end-to-end, runtime, formal, independent, fault, and manual evidence regimes.
- **Specification:** define claims, paths, artifacts, facets, oracles, inference range, privacy, and verdicts.

- **CM1:** retain admissible test and observation configurations, including costly or partially independent ones.
- **CM2:** prefer configurations by risk coverage, fault sensitivity, independence, fidelity, privacy, cost, and recovery value.
- **IC:** select an Instance Constitution or Instance Configuration for one system and release context.
- **IR:** produce an Instance Realization or Implementation Realization of runners, stores, surfaces, monitors, and witnesses.
- **Evidence:** execute and preserve bounded results across test and production environments.
- **Revision:** update claims, tests, profiles, and realization from failures and incidents.

This paper reaches Specification and proposes CM1/CM2 criteria. It does not select an IC, implement CTR, or certify an IR. Realization-stage IR remains distinct from Intermediate Representation in MCIR and other compiler contexts.

14. Evaluation Program

Evaluate the architecture on systems with seeded failures across emission, transport, storage, derivation, display, access, staleness, chain integrity, authority, effect correspondence, and recovery. Compare conventional behavioral CI, ad hoc observability tests, and DSS:OS:TEST profiles.

Measure failure-detection rate, false assurance, time to detection and diagnosis, claim coverage, oracle error, independent-path agreement, staleness, flaky-test influence, environment-transfer failures, bypass detection, incident reconstruction, privacy leakage, operational cost, and recovery success. Use blinded fault injection and mutation where safe and authorized.

Evidence-Path Assurance Profile

Across the completed P1 corpus, Testing for Deriving Observable State is the **evidence-path assurance profile for determining whether a declared system property can be assessed through a bounded chain from event to interpretation**. It treats tests as controlled challenges to evidence production, transport, derivation, access, and recovery rather than as automatic proof of behavior or production readiness.

- **Adjacent work:** *Governance Witnesses*
  Supplies: Typed observation and witness roles with competence and scope limits.
  Observable-state boundary: This paper tests whether those observations traverse a usable evidence path.
- **Adjacent work:** *Reasoning by Way of Receipts*
  Supplies: Claim-bounded inference from retained artifacts.

Observable-state boundary: Test artifacts become premises, not self-validating conclusions.

- **Adjacent work:** *Epistemic Situational Awareness and the Governed Knowledge Loop*
  Supplies: Higher-order orientation from current evidence, uncertainty, and frontier.
  Observable-state boundary: Observable-state tests qualify the inputs; ESA remains a revisable assessment.

- **Adjacent work:** *The Constitutional Observability Platform*
  Supplies: Role-aware presentation of state, claims, effects, warnings, and evidence.
  Observable-state boundary: A platform surface is one path node and may itself be stale, filtered, or inaccessible.

- **Adjacent work:** *Governance Coverage Gaps*
  Supplies: Detection of consequential paths omitted from governance or evidence.
  Observable-state boundary: Evidence-path assurance measures known claim paths without proving unknown-path absence.

An observability claim should bind:

$$\begin{aligned} O = (&subject, property, event, path, oracle, challenge,\\ &environment, independence, latency, access, privacy,.\\ &result, inference, unknowns, expiry) \end{aligned}$$

Every path edge should be tested for omission, fabrication, reordering, duplication, corruption, staleness, access failure, and common-mode control. A complete-looking chain may be circular when the runner, event source, derivation, dashboard, and adjudicator share one producer or trusted base. Independence is therefore a topology and failure-domain claim, not a witness count.

Relative observability closure should report the set of critical claims with adequate current support and the set left missing, stale, circular, or indeterminate. A weighted aggregate can aid prioritization but must not hide one safety-critical blind spot. Deployment readiness additionally requires production correspondence, effect evidence, recovery, and decision authority.

The universal GSM can type
`DECLARES_OBSERVABILITY_CLAIM`, `EMITS_EVENT_EVIDENCE`,
`TRAVERSES_EVIDENCE_PATH`, `CHALLENGES_PATH_EDGE`, `DEPENDS_CIRCULARLY_ON_SURFACE`,
`WITNESSES_INDEPENDENTLY`, `EXPIRES_OBSERVATION`, `LEAVES_CRITICAL_CLAIM_UNKNOWN`,
`SUPPORTS_BOUNDED_INFERENCE`,
and
`DOES_NOT_CERTIFY_PRODUCTION`. These relations give tests an exact epistemic role.

This integration establishes four editorial commitments: retain the paper as the evidence-path assurance profile; adopt the observability-claim envelope and edge-fault taxonomy; represent independence as topology and failure-domain separation; and keep behavioral success, observability support, production correspondence, recovery, and deployment authorization as separate claims.

Claim Decomposition Before Path Assurance

The fixed-seed comparison with *Classified Earthworks*, Project57, *The Phenomenal Aperture*, and the consciousness-disambiguation protocol reveals a prerequisite: evidence-path assurance cannot rescue an ill-specified observable claim.

$$C = (term, sense, target, property, indicator, oracle, path, \\ inference, claim_ceiling, remainder).$$

Before testing a path, the reviewer must distinguish the target property from its indicator and from the conclusion inferred from that indicator. A complete evidence path can faithfully deliver the wrong measurement, and a reliable oracle can decide a proxy that does not support the stated claim. Lexical and target disambiguation therefore precede path assurance without replacing empirical testing.

Project57 provides an instrument case: response scoring, rater agreement, and output custody can be observable even when construct validity remains unresolved. Classified Earthworks provides a physical case: sample and sensor paths can be intact while spatial representativeness, use-class relevance, or site transfer fails. The Phenomenal Aperture provides a presentation case: domain state, rendered surface, human perception, and interpretation are separate path nodes with different failure modes.

For consciousness-related software claims, report behavior, cognitive access, functional organization, self-modeling, context persistence, phenomenality, moral status, and precaution require separate targets and mappings. Observing one does not silently close the evidence path for another.

The universal GSM can type
`DECOMPOSES_OBSERVABILITY_CLAIM`,
`DISTINGUISHES_PROPERTY_FROM_INDICATOR`, `DELIVERS_PROXY_WITHOUT_CONSTRUCT_VALIDITY`,
`SEPARATES_DOMAIN_FROM_PRESENTATION_STATE`, `MAPS_OBSERVATION_TO_BOUNDED_INFERENCE`,
and
`LEAVES_TARGET_REMAINDER_UNRESOLVED`.

This randomized review establishes four editorial commitments: require claim decomposition before path testing; separate property, indicator, oracle, and inference; add physical, psychometric, and presentation transfer failures; and require explicit mappings among distinct machine-consciousness targets.

Effect Correspondence and Common-Mode Failure

Six downstream uses turn evidence-path assurance into a cross-domain profile. Admissibility Engineering depends on evidence rather than behavior alone. COP presents evidence without becoming its source of truth. Constitutional syscalls require reconciliation between requested, dispatched, and observed physical effects. Sovereign Software uses derived state without confusing it with rebuildability or exit. The Ableman Wheel and Classified Earthworks specialize the profile to physical instruments, samples, sites, and monitored outcomes.

The downstream-stable assurance object is:

$$\begin{aligned} A = (&claim, subject, challenge, expected_event,\\ &actual_effect, path, oracle, independent_anchor,\\ &fault_domains, counterfactual, recovery, inference,\\ &expiry, remainder) \end{aligned}.$$

A path can be internally complete and still fail to correspond to the effect. The emitter, runner, transport, derivation, dashboard, and adjudicator may share one producer, clock, schema defect, credential, or interpretation. Assurance must therefore identify common-mode cuts and use independent anchors where the claim consequence warrants them. Independence is graded by shared failure domain, not counted by witness labels.

The profile should challenge both false absence and false presence: suppress a real event, fabricate one, reorder or duplicate it, corrupt its identity, delay it beyond the decision window, deny authorized access, present stale derivation as current, and produce a declared success without the corresponding effect. Recovery evidence should show whether the system can detect, quarantine, reconstruct, reconcile, and revise the claim after such failures.

The universal GSM can type
`BINDS_CLAIM_TO_EFFECT_PATH`, `CHALLENGES_FALSE_ABSENCE`,
`CHALLENGES_FALSE_PRESENCE`, `SHARES_COMMON_FAILURE_DOMAIN`, `ANCHORS_EFFECT_INDEPENDENTLY`,
`RECONCILES_DECLARATION_WITH_EFFECT`, `EXPIRES_OBSERVABILITY_SUPPORT`,
and
`RECOVERS_WITH_REMAINDER`.

This reverse-dependency review establishes four editorial commitments: adopt the effect-correspondence assurance object; model independence by common failure domains; require paired false-presence and false-absence challenges; and make detection, quarantine, reconstruction, reconciliation, expiry, and remainder part of the evidence claim.

15. Counterevidence and Failure Modes

The architecture should be narrowed or rejected where: (1) test evidence is called mathematical proof without a proof system; (2) passing behavior implies observability; (3) observability implies correctness; (4) receipts imply success; (5) chain integrity implies truth; (6) assessments imply causation; (7) surfaces imply complete evidence; (8) tests depend circularly on the surfaces they validate; (9) evidence classes are treated as exclusive; (10) test count replaces adequacy; (11) criticality is unweighted; (12) oracles are wrong; (13) fixtures differ materially from production; (14) nondeterminism is hidden; (15) retries erase failures; (16) flaky tests remain binding; (17) clocks are untrusted; (18) events are dropped before receipt; (19) transport reorders or duplicates; (20) storage corrupts or truncates; (21) derivations lose epistemic metadata; (22) dashboards are stale; (23) access prevents diagnosis; (24) privacy is violated by observability; (25) assert_clean hides components; (26) anomaly thresholds miss attacks; (27) self-tests create circular confidence; (28) the trusted base is undeclared; (29) closure inventory omits critical claims; (30) numerical closure hides a critical gap; (31) deployment rests on tests alone; (32) runtime monitors share one failure mode; (33) evidence paths can be bypassed; (34) effect correspondence is not checked; (35) recovery is untested; (36) GMB inclusion becomes validation; (37) GSM support edges become proof; (38) corpus roles are merged; (39) a specification is presented as implemented; (40) existing practice already supplies the capability; (41) costs exceed diagnostic benefit; or (42) external research contradicts novelty or sufficiency claims.

Test the Evidence Path, Then Test Whether It Corresponds to Reality

Public orientation

Testing for Deriving Observable State treats a test as bounded support for a claim about an evidence path: emission, capture, transport, storage, derivation, presentation, access, and interpretation. A system can behave correctly without useful evidence or emit convincing evidence while behaving incorrectly. Observability closure is always relative to a declared claim inventory and profile.

Adversarial tests

1. **False-absence attack.** A real event is suppressed, dropped, delayed, redacted, inaccessible, or outside retention and the surface reports nothing happened.
2. **False-presence attack.** A fabricated, duplicated, replayed, stale, or misidentified event is presented as a current real effect.
3. **Common-mode attack.** Subject, emitter, runner, transport, derivation, dashboard, and adjudicator share one schema defect, producer, clock, credential, or interpretation.

4. **Dashboard-oracle attack.** A polished surface is treated as the source of truth rather than a projection with its own transformations and omissions.
5. **Test-production transfer attack.** Controlled evidence-path success is generalized across different production load, topology, data, permissions, failures, and adversaries.
6. **Privacy-access attack.** Evidence sufficiency is pursued through excessive retention or access, or privacy controls make closure claims silently incomplete.
7. **Retry-laundering attack.** Repeated challenges eventually pass while failed attempts, nondeterminism, and changed state disappear from the result.
8. **Readiness-collapse attack.** Evidence-path success is treated as deployment readiness, functional correctness, safety, security, or operational fitness.

Claim and evidence ceiling

The paper supplies an evidence-path assurance profile and a constitutional test-artifact contract. It does not prove system knowability, effect correspondence, truthful telemetry, independent witnessing, production behavior, deployment readiness, or complete observability. Tests support only declared claims under exact scenarios, oracles, environments, and inference ranges.

Claims about observability tools, reliability, security, privacy, production transfer, incident outcomes, deployment benefit, or cross-domain assurance require current external sources and exact fault-injection studies with independent anchors. No external research was added in this pass.

Reader-facing consequence

The public question is not simply whether a test passed. It is whether the path from real effect to human conclusion was challenged for missing, forged, stale, inaccessible, and commonly corrupted evidence - and what the result is actually allowed to prove.

17. Conclusion

Testing can expand observable state when it challenges the complete evidence path and records a bounded, interpretable result. This is more demanding than asserting that logs exist or a dashboard is green. It requires explicit claims, independent-enough observations, strong oracles, known inference limits, and continued production validation.

A system that behaves correctly but cannot be diagnosed may not be ready for its intended context. The converse is equally important: a system that can describe itself beautifully may still be wrong. Constitutional verification must preserve both questions and the boundary between them.

Reference and Verification Note

No external source was searched in this pass. Further evidentiary strengthening requires authorized primary-source review of software testing theory, observability, monitoring, distributed tracing, runtime verification, CI/CD, safety and assurance cases, evidence theory, reliability engineering, fault injection, incident response, privacy-preserving telemetry, and existing test-metadata and coverage systems.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Possibility Space.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.
- **Revision.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.

Corpus-Wide Reference Positioning

This paper argues that software tests commonly verify selected behavior under controlled conditions. Its nearest external comparison fields are provenance, software, observation. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Telemetry can be complete at the instrumented boundary while omitting the causal variable, real-world effect, or

authority question that matters. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E2**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

[**E3**] OpenSSF. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2*. 2026. https://slsa.dev/spec/v1.2/ **Relationship:** Adjacent software-supply-chain assurance standard, especially relevant to provenance and verification boundaries.

[**E4**] OpenTelemetry Authors. *OpenTelemetry Specification*. https://opentelemetry.io/docs/specs/otel/ **Relationship:** Adjacent observability standard for traces, metrics, logs, and context propagation; telemetry is not equated with adjudicated evidence.

[**E5**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E6**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E7**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1.* NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

Ableman corpus

[**A1**] Adam Ableman. *Reasoning by Way of Receipts.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-04E427F6F4D94454`;
source SHA-256
`AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *The Epistemic Lineage Engine.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-E0EC3DC7EABF5060`;
source SHA-256
`CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Three Hash Rule.* Ableman Corpus current edition; work identity `PAPER.PUB.20788617`;
source SHA-256
`F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *A Formal Theory of Work Custody.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-C8F2F9BF38C45E29`;
source SHA-256
`A80657E21BA277ADF1C229A2AAE221963B97A62EA2AF3EBC36434A916A8EEA44`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Governance Witnesses.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-9E8DA0EFE860DE5D`;
source SHA-256
`8BF8C3646040F6E1F5222A8AA92C1106AA1F841E8A2C65402F468C83D59D06E5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Sovereign Software.* Ableman Corpus current edition;

work identity
AMCS-LOCAL-WORK-24C6159B98023A9C;
source SHA-256
6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems.* Ableman Corpus current edition; work identity PAPER.PUB.20472196;
source SHA-256
3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848335